

Unit-II: Server-Side Programming with Python (8 hours)

Compiled By

Lecturer, Sunil Bahadur Bist

NAST, Dhangadhi

File Handling in Python

- File handling is an important part of any web application.
- Python has several functions for creating, reading, updating, and deleting files.
- The key function for working with files in Python is the `open()` function.
- The `open()` function takes two parameters; filename, and mode.

File Handling in Python

- There are four different methods (modes) for opening a file:
- "r" - Read - Default value. Opens a file for reading, error if the file does not exist
- "a" - Append - Opens a file for appending, creates the file if it does not exist
- "w" - Write - Opens a file for writing, creates the file if it does not exist
- "x" - Create - Creates the specified file, returns an error if the file exists

File Handling in Python

- In addition you can specify if the file should be handled as binary or text mode
- "t" - Text - Default value. Text mode
- "b" - Binary - Binary mode (e.g. images)

File Handling in Python

- **Syntax**

- To open a file for reading it is enough to specify the name of the file:

```
f = open("demofile.txt")
```

- The code above is the same as:

```
f = open("demofile.txt", "rt")
```

- Because "r" for read, and "t" for text are the default values, you do not need to specify them.

File Handling in Python

demofile.txt

Hello! Welcome to demofile.txt

This file is for testing purposes.

Good Luck!

```
f = open("demofile.txt")
```

```
print(f.read())
```

File Handling in Python

- If the file is located in a different location, you will have to specify the file path, like this:

Example

- Open a file on a different location:

```
f = open("D:\\myfiles\\welcome.txt")  
print(f.read())
```

File Handling in Python

- **Using the with statement**
- You can also use the with statement when opening a file:
 with open("demofile.txt") as f:
 print(f.read())

File Handling in Python

- **Close Files**

- It is a good practice to always close the file when you are done with it.
- If you are not using the with statement, you must write a close statement in order to close the file:

Example

- Close the file when you are finished with it:

```
f = open("demofile.txt")  
print(f.readline())  
f.close()
```

File Handling in Python

- **Read Only Parts of the File**
- By default the `read()` method returns the whole text, but you can also specify how many characters you want to return:

Example

- Return the 5 first characters of the file:

```
with open("demofile.txt") as f:  
    print(f.read(5))
```

File Handling in Python

- **Read Lines**

- You can return one line by using the `readline()` method:

Example

- Read one line of the file:

```
with open("demofile.txt") as f:  
    print(f.readline())
```

File Handling in Python

- By calling `readline()` two times, you can read the two first lines:

Example

- Read two lines of the file:

```
with open("demofile.txt") as f:
```

```
    print(f.readline())
```

```
    print(f.readline())
```

File Handling in Python

- By looping through the lines of the file, you can read the whole file, line by line:

Example

- Loop through the file line by line:
 with open("demofile.txt") as f:
 for x in f:
 print(x)

File Handling in Python

- **Write to an Existing File**
- To write to an existing file, you must add a parameter to the `open()` function:
- `"a"` - Append - will append to the end of the file
- `"w"` - Write - will overwrite any existing content

File Handling in Python

- Open the file "demofile.txt" and append content to the file:
 with open("demofile.txt", "a") as f:
 f.write("Now the file has more content!")
- #open and read the file after the appending:
 with open("demofile.txt") as f:
 print(f.read())

File Handling in Python

- **Overwrite Existing Content**

- To overwrite the existing content to the file, use the w parameter:

Example

- Open the file "demofile.txt" and overwrite the content:

```
with open("demofile.txt", "w") as f:
```

```
    f.write("Woops! I have deleted the content!")
```


File Handling in Python

- #open and read the file after the overwriting:

```
with open("demofile.txt") as f:  
    print(f.read())
```

Example

- Create a new file called "myfile.txt":

```
f = open("myfile.txt", "x")
```

File Handling in Python

- **Delete a File**
- To delete a file, you must import the OS module, and run its `os.remove()` function:
- Remove the file "demofile.txt":

```
import os  
os.remove("demofile.txt")
```

File Handling in Python

- **Check if File exist:**
- To avoid getting an error, you might want to check if the file exists before you try to delete it:

Example

- Check if file exists, then delete it:

```
import os
if os.path.exists("demofile.txt"):
    os.remove("demofile.txt")
else:
    print("The file does not exist")
```

File Handling in Python

- **Delete Folder**

- To delete an entire folder, use the `os.rmdir()` method:

Example

- Remove the folder "myfolder":

```
import os
```

```
os.rmdir("myfolder")
```

Working with CSV and JSON Files

- Working with CSV and JSON files in Python involves using the `csv` and `json` libraries to read, write, and manipulate these structured data formats.
- CSV (Comma Separated Values) is a simple text format for tabular data,
- while JSON (JavaScript Object Notation) is a more flexible format for storing structured data.
- Both are commonly used for data storage, transfer, and analysis.

Working with CSV and JSON Files

Example of writing to a CSV file

```
import csv
```

```
with open('output.csv', 'w', newline='') as file:
```

```
    csv_writer = csv.writer(file)
```

```
    csv_writer.writerow(['name','email','mobile']) # Write header
```

```
    csv_writer.writerow(['sunil','sunil@nast.edu.np','9858585858']) #data
```

```
    csv_writer.writerow(['ravi', 'ravi@nast.edu.np', '9858585859']) #data
```

```
    csv_writer.writerow(['pusp', 'pusp@nast.edu.np', '9858585860']) #data
```

Working with CSV and JSON Files

```
# Example of reading a CSV file
import csv
with open('output.csv', 'r') as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        print(row)
```

Working with CSV and JSON Files

Example of writing to a JSON file

```
import json
```

```
data = {'name': 'Sunil', 'age': 35, 'city': 'Dhangadhi'}
```

```
with open('output.json', 'w') as file:
```

```
    json.dump(data, file, indent=4) # 'indent' for pretty-printing
```


Working with CSV and JSON Files

Example of reading a JSON file

```
import json
```

```
with open('output.json', 'r') as file:
```

```
    data = json.load(file)
```

```
    print(data)
```